

Per%Sense

User-Interface Comparison: DOS Original vs. the Go Web Port

*A screen-by-screen walkthrough of how each calculator is operated,
with the Windows port as the visual reference point.*

Prepared June 23, 2026

Contents

1. How to read this document

Per%Sense has lived through three user interfaces. The **original DOS program** presented every calculator as a full-screen, 80×25 text display driven by a menu bar and function keys. It was later re-skinned as a **Windows desktop application** built from native form windows. The current work is a **Go web port** — a single-page web application served by one self-contained program.

Throughout this project two principles govern which legacy version is authoritative, and they are worth stating up front because they explain almost every difference below:

- **The DOS version is the authority for what each calculation does** — the financial logic, rounding and edge-case behavior. The web port reproduces DOS results, not the Windows ones, wherever the two ever disagreed.
- **The Windows version is the authority for how things should look** — layout, labels, the input-vs-computed colour convention. The web port modernizes that look rather than copying it pixel-for-pixel.

Each screen below is presented the same way: a short description, a four-column table comparing the **DOS**, **Windows** and **Go web** experience, and a note on what changed and why. The final sections cover the cross-cutting themes — navigation, the fill-in-the-blank input model, errors, help, and output — plus one calculation difference that is visible on screen.

2. The shift in interaction model, at a glance

Before the screen-by-screen detail, three structural shifts shape everything else:

From one screen at a time to a navigable app

DOS: one calculator filled the whole screen; you switched screens through the menu bar and function keys.

Windows: a multi-document (MDI) parent window hosting child windows — Welcome, Mortgage, Amortization, Present Value and the APR Report could each be open at once.

Go web: a single-page app with a persistent top navigation bar (Home · Mortgage · Amortization · Present Value). One worksheet shows at a time, but your entries on every screen are kept as you move between them.

The fill-in-the-blank model survives unchanged

All three versions share Per%Sense's signature idea: **you type the values you know and leave blank the one you want solved for**. The program detects the empty field and computes it. White cells are inputs; computed answers come back in a distinct colour. This model is preserved exactly in the web port.

Modern web affordances are added, not subtracted

The web port keeps the legacy model but layers on things the DOS and Windows versions never had: a task-oriented welcome screen, automatic saving of your worksheet between visits, CSV export and printing, an interactive tour, per-field help tooltips, dark mode, mobile-responsive layout, and inline error highlighting.

3. Welcome / home screen

The entry point. In the desktop eras it was a launcher; in the web port it doubles as a guide that points you at the right calculator by the question you are trying to answer.

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
Purpose	Title screen; pick a worksheet via the menu/function keys.	A small fixed window titled “Per%Sense — Every financial calculation you can imagine,” with three worksheet buttons plus Help and Examples.	A full landing page that routes you by task, not just by calculator name.
How you choose	Menu bar + function keys.	Mortgage / Amortization / Present Value buttons, colour-coded (green / yellow / magenta).	Plain-language question buttons grouped under Mortgage, Amortization, Present Value and Life Contingency — e.g. “What’s my monthly payment?”, “How much do extra payments save?”, “What’s a lifetime pension worth?”
Getting started	Manual; consult the printed manual.	Help and Examples buttons.	Take the Tour (interactive), Load Examples, and Import .psn File for opening a saved worksheet.
Look & feel	Text, box-drawing borders, colour attributes.	Native Windows window, fixed size.	Responsive page, light/dark theme toggle.

What changed and why: the web welcome screen turns the launcher into onboarding. Rather than asking you to already know that an annuity valuation is a “Present Value” task, it lists the everyday questions each calculator answers and loads the right screen pre-aimed at that question.

4. Mortgage screen

A grid where each row is one mortgage scenario. You fill in the figures you have — price, percent down, rate, term, monthly payment — and leave blank the one you want the program to find.

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
Layout	Full-screen text grid; columns scroll.	A 12-column grid (Price, Points, % Down, Cash Required, Amount Borrowed, Years, Loan Rate, Monthly Tax, Monthly Total, Balloon Years, Balloon Amount) with a summary box above the current row.	The same columns as an HTML grid that grows rows as needed, with three-state column sorting.
Input vs. answer	Colour attributes distinguish entered from computed.	White input cells; “money-green” computed cells; the selected cell is yellow.	White inputs; computed cells are green and also carry an accent border, italic and bold weight (so the cue is not colour-only).
Actions	Function keys; menu	Toolbar / menu: Calculate,	Toolbar: Calculate Row,

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
	Calculate.	Cut/Copy/Paste, Undo/Redo, Compare APRs, Generate Rows.	Calculate All, Compare APR, What-If Table, Export CSV, Clear Row, Clear All.
Reusing an answer	—	—	“Harden”: double-click (or press H) a computed cell to lock it as an input for the next solve.
Scenario sweeps	Row generation.	Generate Mtg Rows dialog (vary a column by an increment over N rows).	What-If Table dialog — vary one or two fields across a range and fill the grid.

What changed and why: the model and columns are carried straight over from the Windows grid, including the white-input / green-output convention. The web port adds column sorting, CSV export, a richer two-dimensional What-If, and inline highlighting of the cell that caused an error. The desktop Undo/Redo is the main capability not yet reproduced on the web.

5. Amortization screen

Builds a full payment schedule for a loan. A handful of top-level fields define the loan; an Advanced Options area layers on real-world complications — extra payments, balloons, rate changes, payment holidays and more.

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
Loan inputs	Text fields for amount, dates, rate, term, payments/year, payment.	Top-of-screen fields: Amount, Loan Date, Loan Rate, First Date, Periods, Last Date, Per Year, Payment, Points, APR.	The same fields as single inputs across the top; leave one blank to solve for it.
Advanced options	Available via the option blocks.	Separate grids for Payoff, Balloons, Prepayments (start/stop/frequency/amount), Rate Adjustments (ARMs), Moratorium, Target principal, and Skip Months.	A collapsible “Advanced Options” panel holding the same option tables; each table has a “+ Add Row.”
Run it	Menu Calculate.	Calculate; Table Output dialog (detail/summary; to screen, CSV, text or spreadsheet).	“Calculate” builds the schedule on screen; Export CSV; Clear All (with confirmation).
Schedule output	Text table / print.	On-screen table plus the Table Output export formats.	On-screen schedule table; editing any input clears the now-stale schedule so the numbers never lag the inputs.

What changed and why: the Advanced Options set is reproduced in full (prepayments, balloons including off-cycle dates, ARM adjustments, moratorium, target, skip-months). The web port folds the Windows “Table Output” chooser into a simpler on-screen table plus CSV export, and adds automatic invalidation of a stale schedule when you change an input.

6. Present Value screen

Values streams of money across time — lump sums and periodic payments — back to a single “as-of” date. It also hosts the most specialized features: variable interest-rate schedules and life-contingency (actuarial) valuation.

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
Core inputs	As-of date, rate, and the cash-flow rows in text blocks.	A Present-Value line (As-of, rate, value, duration); a Lump-Sum grid (Date / Amount / Value); a Periodic grid (From / To / Per-Year / Amount / COLA / Value).	As-of date, rate-type selector, rate and a Total field (blank = compute) across the top; Lump-Sum and Periodic grids below, each with “+ Add Row.”
Variable rates	“Fancy” mode adds a rate-schedule block.	Advanced/“Fancy” mode reveals a Rate-line grid (effective date + rate) and an extended valuation block.	A collapsible “Variable Rate” section holding the rate schedule (date + rate).
Life contingency	Driven by a “Life” column (letters N / L / D / 1 / 2 / E / B) and a payment-on-death amount.	Same “Life” column on each row plus the payment-on-death input.	A collapsible “Actuarial (Life Contingency)” section, including payment-on-death; contingent rows show the survival probability applied.
Run it	Menu Calculate.	Calculate.	Calculate; Export CSV; Clear All.

What changed and why: the cash-flow grids and the life-contingency “Life” column carry over. Rather than a single “Advanced” toggle, the web port breaks the specialized features into two clearly labelled, independently collapsible sections (Variable Rate and Actuarial), so each is discoverable on its own. One small label note: the Windows periodic column reads “Through” where the web port and code call it “To Date.”

A note on the life-contingency feature

The actuarial valuation (annuities “for life,” payment-on-death) was a real shipped feature, but its computational core and mortality tables are not present in the source materials we received. The web port reconstructs this math and validates it against an independent, published actuarial standard rather than against the original program. Functionally the screen behaves as before; the one thing that cannot be machine-confirmed from the current materials is bit-for-bit equality with the original's specific mortality table. (Supplying the original actuarial module would close that gap.)

7. APR comparison & report

Compares two mortgage options on a true annual-percentage-rate basis, accounting for points and closing costs, and produces a lender-style report.

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
Setup	Enter points/costs/payments; pick rows to compare.	APR Options dialog (Points, Costs, Payments) and a Select-APR dialog to choose which rows to compare.	Driven from the Mortgage screen's “Compare APR” action.
Result	Text comparison.	APR Comparison dialog showing APR 1, APR 2 and a	An on-screen comparison of the two APRs.

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
		plain-language verdict (e.g. which option is better).	
Report	Printed report.	A dedicated APR Report window (payment structure, APR, finance charges, totals) with print.	Print via the page's print stylesheet.

8. Settings

The computational settings change how the math is performed (day-count basis, interest rule, prepaid first period, and so on). All three versions expose essentially the same set.

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
Computational	Settings screens.	A Computational Settings dialog: payments/year, COLA month, USA vs. Actuarial rule, day-count basis, prepaid first period, interest in arrears/advance, balloon handling, Rule-of-78, exact mode.	A Settings modal with the same options; a count badge appears whenever a setting differs from its default, so an active non-default rule is visible.
Appearance	Colour attributes.	UI Settings dialog: cell colours and fonts. Clipboard delimiters dialog.	Light/dark theme; input/computed cell colours are set by the theme.
Obsolete / inactive	—	"Year to divide century" (a 1990s two-digit-year setting).	Retained but disabled, with a tooltip explaining it has no effect; a couple of never-wired frequency options (Canadian/Daily) are hidden rather than shown as dead controls.

9. Cross-cutting differences

9.1 Entering data

DOS accepted typed values with function-key navigation; Windows used native grid controls with dates typed as plain text. The web port keeps text entry but adds guidance: date fields auto-insert the MM/DD/YYYY slashes as you type, numeric fields request the numeric keypad on mobile, and a "t" shortcut fills today's date. One deliberately non-standard choice carried from the desktop model: **Enter moves to the next field rather than submitting the form** — efficient for fast data entry, but unusual for a web page.

9.2 Errors

Where DOS used a status line and Windows used pop-up message dialogs, the web port shows specific, actionable messages (it names the offending row or field and suggests a fix), highlights the problem cell inline, and announces errors to assistive technology. Over-determined inputs — e.g. supplying both Price and Monthly Total when only one can be free — are caught and explained.

9.3 Help

The web port's help is its standout: layered from a welcoming home screen, to a one-line hint on each screen, to a “?” tooltip on nearly every field, to a full help page with a worked example for every screen, to an interactive first-run tour. Contextual Help buttons deep-link to the relevant section. This goes well beyond the desktop help system.

9.4 Saving, output and printing

The Windows version saved and opened worksheet files and offered several table-output formats. The web port automatically saves your worksheet in the browser and restores it on your next visit, can import the legacy .psn worksheet files, exports CSV from all three calculators, and provides a clean print layout. The desktop file-based Save/Open and multi-format Table Output chooser are simplified into autosave + CSV + print; multi-level undo/redo is the main desktop capability not yet reproduced.

9.5 Accessibility and reach

Entirely new in the web port and not applicable to the earlier versions: keyboard and screen-reader support (ARIA roles, focus management in dialogs), a dark theme, and a layout that adapts to phone-sized screens. The program also now runs anywhere a browser does, from a single downloadable file, with no installation.

10. DOS-era performance patterns that no longer apply

A number of techniques in the original program were not design choices so much as concessions to 1990s hardware. A DOS application ran on a slow processor, with no graphics accelerator and no operating-system layer between it and the screen, so to feel responsive it had to address the hardware directly and avoid any work it could. The Go web port inherits *none* of this machinery — the browser, the operating system, and the Go runtime now do what the DOS program had to do by hand. The patterns below are listed not because anything was wrong with them, but because they explain a whole layer of the original that has no counterpart in the port, and needs none.

DOS technique	Why it existed (1990s hardware)	Why the Go web port drops it
Writing straight to video memory	Characters and their colours were written directly into the display adapter's text buffer (segment \$B800 colour, \$B000 mono) through a screen pointer, bypassing DOS and the BIOS — the only way to update the screen fast enough.	The browser and operating system own the screen. The port changes the page and the system repaints it; no application writes to display memory.
The 80×25 character/attribute grid	Every screen cell was exactly two bytes — a character plus a packed colour attribute — so the whole screen was 4,000 bytes and could be repainted with a single block copy.	The web UI has no fixed grid and no attribute bytes; each element is styled independently with CSS and laid out fluidly. Byte-level compactness is irrelevant.
Snapshot-and-restore for pop-ups	To show a menu or dialog, the program copied the raw screen bytes aside and blitted them back afterward (MoveScreen), because redrawing the screen underneath was too costly.	A retained-mode UI redraws itself automatically; a dialog is just another element layered on top. There is nothing to snapshot or restore.

DOS technique	Why it existed (1990s hardware)	Why the Go web port drops it
CGA “snow” suppression	On early CGA cards, writing to video memory outside the display's retrace interval produced visible static, so the program detected a CGA card and timed its writes to avoid it.	A pure artifact of 1980s analog display hardware. No modern display exhibits it, so the detection and timing code is meaningless.
Hand-managed segmented memory	16-bit DOS addressed memory as segment/offset pairs; the program normalized and advanced pointers by hand and overlaid integers on byte arrays (“absolute” aliasing) to move and compare several bytes at once.	Go uses a flat memory model with garbage collection and optimizes such access itself. Manual pointer arithmetic would be both unsafe and pointless.
Going straight to BIOS / DOS	Keyboard modifier state and the video mode were read directly from the BIOS data area, and the date came from a DOS interrupt (INT 21h) — the most direct path to the hardware.	The port receives keyboard and date information through the browser and the Go standard library; it never touches interrupts or hardware registers.
Direct text-cursor control	The blinking block cursor was positioned, resized and hidden by programming the display hardware's cursor registers directly.	The browser manages the text caret. Cursor handling is no longer the application's concern.
Paged “blocks” of data	Because a text screen cannot cheaply scroll arbitrary content, long schedules and row lists were shown one fixed window — a “block” — at a time.	The web page scrolls natively and can render an arbitrarily long schedule or grid as one continuous, scrollable view.

The net effect is that an entire stratum of the original program — the part that coaxed a fast, flicker-free display out of limited hardware — simply does not exist in the port, and did not need to be re-created. What was painstaking hardware-level drawing code in the DOS version is, in the web version, declarative markup that the browser renders. The effort that once went into *painting* the interface is now spent entirely on its behavior and clarity.

11. One calculation difference you can see on screen

Almost every difference in this document is about presentation. There is one place where the *numbers* differ between the legacy versions, and because the web port follows the DOS authority it can show a different figure from the old Windows help screens.

When a loan's **first period is “odd”** — the first payment falls a short or long gap from the loan date rather than exactly one period later — the regular payment must be nudged so the loan still pays off over the stated number of payments. The authoritative DOS engine makes this adjustment; the old Windows help screens displayed the un-adjusted payment. The web port matches DOS.

Aspect	Original DOS (text-mode)	Windows port (GUI)	Go web port
Regular payment	\$731.98 (adjusted)	\$733.76 (un-adjusted, per help)	\$731.98 — follows DOS
Total interest	\$163,513.81	\$161,499.77	\$163,513.84 — follows DOS

Example: a \$100,000 loan at 8% over 360 monthly payments, dated Feb 12, 2024 with the first payment March 1, 2024 (a 19-day short first period). A natural first period gives the same \$733.76 in all versions; only the odd first period triggers the adjustment.

12. Summary

The web port is best understood as a faithful re-housing of Per%Sense, not a redesign of what it computes. The fill-in-the-blank worksheet model, the column layouts, and the input-vs-computed colour convention are all carried across from the Windows look; the financial results are held to the DOS original. On top of that foundation the web version adds the conveniences a modern user expects — task-oriented onboarding, automatic saving, export and print, layered help and an interactive tour, error highlighting, accessibility, dark mode and a responsive layout — while a small number of desktop-era controls were dropped as obsolete (the century-divide setting) or hidden until wired (a couple of payment-frequency options). The principal desktop capability still on the list is multi-level undo/redo.

Net effect: a returning Per%Sense user will recognize every screen and operate it the same way, while a newcomer is guided in by plain-language questions, help at every field, and a tour — with the assurance that the answers on screen reproduce the original DOS program.